Documentación técnica

Escena 3D: La Revolución Mexicana

Alumno	Edgar Humberto Del Campo Valdés
Número de cuenta	316532215
Asignatura	Laboratorio de Computación Gráfica e Interacción Humano Computadora
Examen	Examen Extraordinario de Laboratorio 2027-1
Fecha de entrega	31 de octubre de 2026
Video demostrativo	https://youtu.be/PMIuDqKpBuY
Repositorio	https://github.com/Boro739/Extraordinario2027-1-CG

1. Descripción general

El proyecto es una escena 3D en tiempo real, desarrollada en C++ con OpenGL 3.3, ambientada en un pueblo de la Revolución Mexicana. Parte del proyecto base proporcionado por el profesor (`github.com/joserapath23/Extraordinario2027-1`), sobre el que se implementaron las seis actividades del examen: iluminación direccional configurable, animación del zorro, integración de modelos temáticos con animaciones, tres spotlights RGB controlables, retexturizado del piso con ajuste de UVs y acomodo de los árboles.

Tecnologías

Componente	Uso
Visual Studio 2022 (MSVC v143), C++, plataforma x86	Compilación del proyecto
OpenGL 3.3 core + GLSL 330	Render y shaders de iluminación
GLFW	Ventana, contexto OpenGL y entrada de teclado/mouse
GLEW	Carga de funciones de OpenGL
GLM	Matemáticas (vectores, matrices, transformaciones)
Assimp 4.1	Carga de modelos OBJ, FBX y GLB, incluidos esqueletos y animaciones
SOIL2 / stb_image	Carga de texturas (archivos e incrustadas en los modelos)

Archivos modificados

Archivo	Cambios
`Main.cpp`	Luz direccional, spotlights, zorro, árboles, modelos y animaciones, teclado
`Shader/lighting.frag`	3 spotlights activas y color de material para modelos sin textura
`Shader/lamp.frag`	Color de los cubos que marcan cada spotlight
`Model.h`, `Mesh.h`	Normales generadas, color de material, texturas incrustadas, caja envolvente
`ModelsExtra/Floor.obj`, `Floor.mtl`	4 materiales y UVs recalculadas
`ExtraOrdinario.vcxproj`	Release compila SOIL2 desde su código fuente

2. Actividad 1: modificación de la iluminación

El proyecto base mezclaba un ambiente "nocturno" rojo `(1, 0, 0)` con un factor de día de 0.5, por lo que toda la escena se veía rojiza. Se reemplazó por una luz direccional cuyo color se elige entre dos constantes y se escala por una intensidad fija de **0.5**:

```
glm::vec3 dirColor = luzNaranja ? colorNaranja : colorBlanco;   // (1,0.5,0) o (1,1,1)
glm::vec3 ambient  = dirColor * intensidadDir * 0.6f;            // intensidadDir = 0.5
glm::vec3 diffuse  = dirColor * intensidadDir;
glm::vec3 specular = dirColor * intensidadDir * 0.3f;
```

- Al iniciar, la luz es **blanca tenue**.
- La tecla **L** alterna entre **blanco** y **naranja**, ambos con intensidad 0.5.
- La dirección se inclinó ligeramente, a `(-0.3, -1, -0.4)`, para que las caras laterales de los modelos tengan sombreado.

3. Actividad 2: integración y animación del zorro

El zorro de `ModelsExtra/Zorro` está dividido en 7 piezas (cuerpo, cabeza, cola y cuatro patas) que vienen ya en su posición final. Se cargan como modelos independientes y se dibujan con una **jerarquía de transformaciones**:

1. Una **matriz base** común con la posición en el carril, la orientación, la inclinación del salto y la escala (0.2). Además mueve el centro del cuerpo al origen y apoya las patas en `y = 0`.
2. Cada pieza se rota sobre su propio **pivote** con `T(pivote) · R · T(−pivote)`:

Pieza	Pivote (coord. del modelo)	Rotación
Patas delanteras	(0, −0.49, −0.915), unión con el cuerpo	eje X
Patas traseras	(0, −0.49, −1.515)	eje X
Cabeza	(0, 0, −0.67), cuello	eje X (cabeceo)
Cola	(0, −0.05, −1.79), base	eje Y (balanceo) y eje X (salto)

Caminata (tecla Z)

- Z inicia o detiene la caminata. El zorro recorre un carril libre de obstáculos, en `z = −4` entre `x = −3` y `x = 5`, sobre la zona plana del terreno y a más de 4 unidades de cualquier árbol.
- Las patas se mueven como un **trote**: los pares diagonales van en fase opuesta, con `30° · sin(ciclo)`. La cola se balancea y la cabeza cabecea ligeramente; el cuerpo tiene un pequeño rebote.

- Al llegar a un extremo, **gira 180° en su lugar** (a 180°/s) y regresa.
- Al detenerlo, la amplitud del paso baja suavemente a 0 y las patas regresan a su posición de reposo.

Salto (tecla X)

- Hace un arco parabólico `y = 4 · h · t · (1 − t)`, con altura `h = 0.5` y duración de 0.9 s.
- Las patas delanteras se estiran hacia adelante y las traseras hacia atrás (±45°), y la cola se levanta.
- El cuerpo se inclina con `−20° · sin(2πt)`: nariz arriba al despegar, abajo al caer y nivelado al aterrizar.
- Puede saltar parado o mientras camina. Si ya está en el aire, la tecla se ignora.

Toda la actualización se hace en `AnimationKeys()` con `deltaTime`, así que la velocidad no depende de los FPS.

4. Actividad 3: integración de modelos adicionales

Se integraron **15 modelos diferentes**, todos descargados de internet y relacionados con un pueblo de la Revolución Mexicana: una estación de tren con locomotora de vapor, la iglesia con su campanario, casas, un cañón, caballería, una Adelita y un jefe revolucionario. Las fuentes y licencias están en las Referencias.

#	Modelo	Autor / fuente	Licencia	Ubicación (x, z)	Animación
1	Locomotora de vapor	Quaternius / Poly Pizza	CC0	vía en z = −1, x de 17 a 41	Bucle 1
2	Vagón de pasajeros	Quaternius / Poly Pizza	CC0	enganchado a la locomotora	Bucle 1
3	Campana	Quaternius / Poly Pizza	CC0	campanario (8, −24)	Bucle 2 (C)
4	Cañón	Quaternius / Poly Pizza	CC0	(−4, −13)	Bucle 3
5	Bandera	Quaternius / Poly Pizza	CC0	(1, −14)	Procedural
6	Jefe revolucionario ("The Boss")	Mixamo (Adobe)	Uso libre de Mixamo	(4.5, −9)	Huesos
7	Adelita ("Peasant Girl")	Mixamo (Adobe)	Uso libre de Mixamo	(−1, −7)	Huesos (adicional)
8	Catedral	Bruno Oliveira / Poly Pizza	CC BY 3.0	(2, −26)	—
9	Casa (2 instancias)	Quaternius / Poly Pizza	CC0	(11, −18), (−1.5, −19.5)	—
10	Caballo	Quaternius / Poly Pizza	CC0	(8, −13)	—
11	Barril (3 instancias)	Quaternius / Poly Pizza	CC0	(6.5, −16)	—
12	Sombrero	Poly by Google / Poly Pizza	CC BY 3.0	sobre un barril	—
13	Rifle	CreativeTrio / Poly Pizza	CC0	junto a los barriles	—
14	Nopal (4 instancias)	Poly by Google / Poly Pizza	CC BY 3.0	(6, 4), (14, −14), (−8, −4), (20, −8)	—
15	Camino de tierra	Poly by Google / Poly Pizza	CC BY 3.0	(16, −6)	—

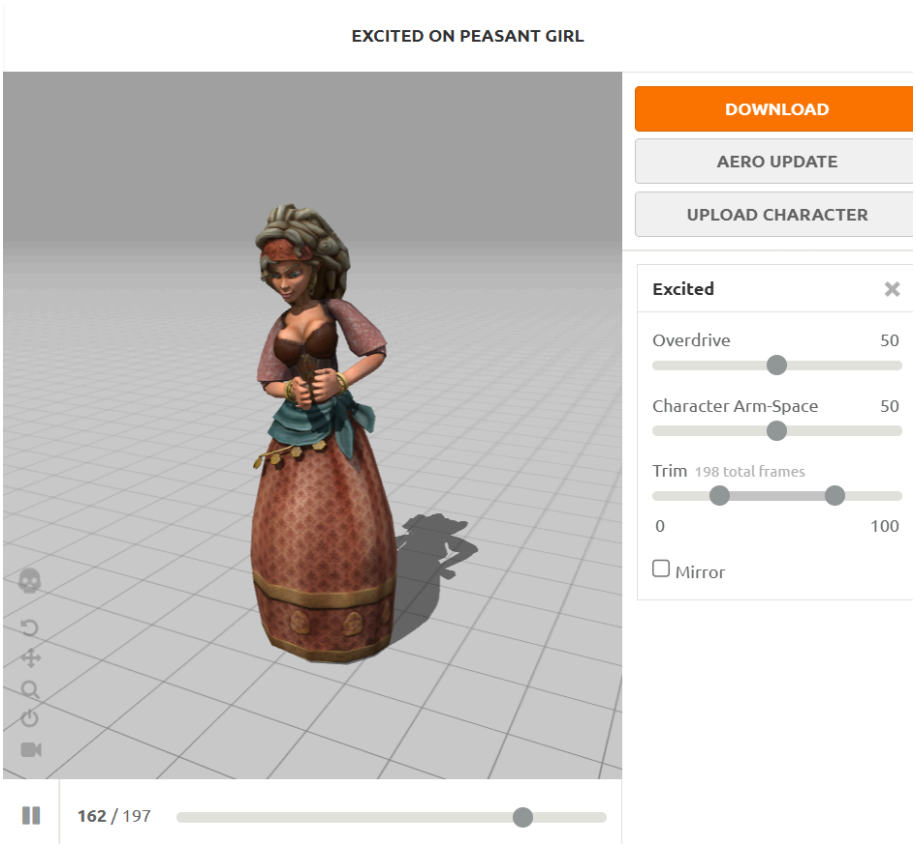
Integración

- **Conversión de formato:** la versión de Assimp del proyecto (4.1) no lee los GLB generados con FBX2glTF; los cargaba con 0 mallas. Por eso esos 9 modelos se convirtieron a OBJ + MTL con un script propio en Python. El script recorre la jerarquía de nodos del glTF aplicando sus transformaciones, conserva normales y UV, y exporta los colores de material

(`baseColorFactor`) y las texturas incrustadas. Los archivos convertidos están en `ModelsExtra/Convertidos/` y los originales en `ModelsExtra/ModelosPolyPizza/`.

- **Colocación:** cada modelo se coloca con `ColocarModelo()`, que lo escala según su dimensión mayor, lo centra y lo apoya sobre el terreno usando `AlturaTerreno(x, z)`. Así ninguno flota ni se hunde. Los objetos estáticos están en el arreglo `objetosEscena[]`.
- **Personajes con huesos:** el esqueleto de Mixamo trabaja en centímetros y su caja envolvente en reposo no coincide con la de la malla. Por eso `MedirConHuesos()` aplica el *skinning* en CPU a la primera pose y mide la caja real antes de escalarlos.
- **Escenografía adicional:** la vía del tren (durmientes y rieles) y la estructura del campanario se dibujan con el cubo del proyecto, escalado y con color de material.

Evidencia de la fuente: descarga de la Adelita ("Peasant Girl") con la animación *Excited* desde Mixamo. El modelo se exportó en formato Collada (.dae) con su esqueleto (*With Skin*) a 30 fps.



Animaciones

Requisito	Modelo	Tipo	Descripción
Bucle continuo 1	Locomotora + vagón	Por código	El tren recorre la vía de ida y vuelta con `x = 29 + 12·sin(0.3·t)`. El movimiento senoidal hace que frene suavemente en cada extremo antes de regresar.
Bucle 2, con pausa/reanudación (C)	Campana	Por código	Oscila como péndulo, colgada de la viga del campanario: `ángulo = 30°·sin(π·t)`, con periodo de 2 s. **C** detiene su reloj de animación (pausa) y lo vuelve a correr (reanuda), así que continúa exactamente desde donde se pausó.
Bucle continuo 3	Cañón	Por código	Cada 3 s "dispara": retrocede 0.35 unidades en 0.1 s y regresa lentamente a su posición en 1.2 s.
Procedural	Bandera	Procedural (vertex shader)	Cada vértice de la tela se desplaza con una suma de funciones seno que depende del tiempo y de su distancia al asta (`sin(6d − 5t) + 0.4·sin(11d − 8t + 1.3)`). El borde del asta queda fijo, la onda viaja hacia la punta y el asta y la base no se mueven.
Por huesos	Jefe revolucionario	Esqueleto (skinning)	Animación *Reloading* de Mixamo (formato Collada) en bucle, reproducida con las clases `Animation`, `Bone` y `Animator`. Las matrices finales de los huesos se envían a `finalBonesMatrices[]` del vertex shader.
Adicional	Adelita	Esqueleto (skinning)	Animación *Excited* de Mixamo en bucle.

> El personaje Steve del proyecto base se dejó en el código como referencia (`mostrarSteve = false`), pero no se dibuja porque no corresponde a la temática. Su animación no se considera para el examen.

5. Actividad 4: spotlights RGB

Las tres spotlights iluminan al **jefe revolucionario** (modelo "The Boss" de Mixamo, con su animación por huesos *Reloading*). Está en la plaza del pueblo, en la posición **(4.5, 0, −9)**: adelante a la derecha de la cámara inicial, entre los árboles y frente a la catedral.

Luz	Tecla	Posición	Color difuso
Roja	R	(4.5, 3.0, −7.0), al frente	(2, 0, 0)
Verde	V	(2.77, 3.0, −10.0), atrás a la izquierda	(0, 2, 0)
Azul	B	(6.23, 3.0, −10.0), atrás a la derecha	(0, 0, 2)

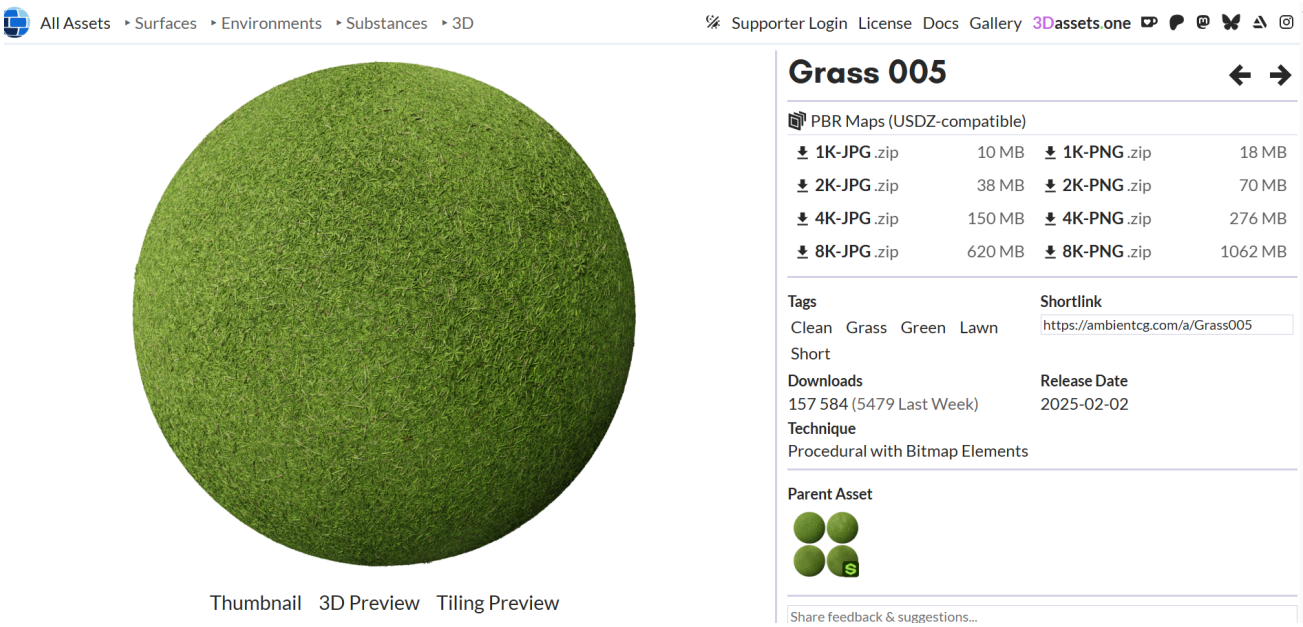
- Las tres están a 120° entre sí alrededor del personaje y a 3 unidades de altura. Su dirección se calcula como `normalize(objetivo − posición)`, con objetivo en `(4.5, 0.8, −9.0)`.
- Cono: `cutOff = cos(15°)` y `outerCutOff = cos(22°)`, con borde suave. Atenuación: constante 1, lineal 0.09, cuadrática 0.032.
- **Modificación del shader:** en `lighting.frag` se cambió `NUMBER_OF_SPOT_LIGHTS` de 2 a 3 y se activó el ciclo que suma `CalcSpotLight()` al resultado, que en el proyecto base estaba comentado.
- R, V y B alternan cada luz en tiempo real. Una luz apagada se envía al shader con color negro.
- Cada luz se marca con un pequeño cubo de su color, dibujado con `lampShader`. El cubo se pone gris cuando la luz está apagada.
- Donde coinciden los tres conos, la suma de rojo, verde y azul da luz blanca.

6. Actividad 5: ajuste de UVs y texturas del piso

`Floor.obj` es una malla de 100 × 100 unidades con 16 × 16 cuadros (256 caras). El centro es plano y hay cerros de hasta 9.3 unidades en las orillas. Originalmente tenía un solo material. Se separó en **4 materiales sin modificar vértices ni normales** (se verificó comparándolo contra el archivo original), así que las alturas usadas para los árboles y el zorro siguen siendo válidas.

Material	Textura	Caras	Criterio de acomodo	Proyección UV
Pasto	`pasto.jpg` (Grass005)	131	Zona plana y lomas suaves	Planar XZ, se repite cada 5 u
Agua	`agua.jpg` (Ice002)	6	Lago de 3 × 2 cuadros en una zona plana	Planar XZ, cada 8 u
Arena	`arena.jpg` (Ground054)	14	Orilla alrededor del lago	Planar XZ, cada 4 u
Tierra	`tierra.jpg` (Ground111)	105	Cerros altos (más de 3 u) y un camino hacia el lago	Tipo caja, cada 5 u

Evidencia de la fuente de las texturas: las cuatro se descargaron de ambientCG, con licencia CC0, en su versión 1K-JPG; de cada paquete se usó el mapa *Color*. La imagen muestra la página de la textura de pasto Grass005.



Las coordenadas UV se calcularon a partir de la posición de cada vértice dividida entre el tamaño de repetición. Así la textura conserva su escala real en todo el terreno y no se estira. Para la tierra se usó una **proyección tipo caja**: según el eje dominante de la normal de cada cara se proyecta sobre XZ, ZY o XY, lo que evita el estiramiento en las pendientes de los cerros. Cada cara recibe sus propias UV (1024 coordenadas en total).

```
Mapa del piso (vista superior; P = pasto, T = tierra, R = arena, A = agua)
T T T P P P P P P P T T T T T
T T T P P P P P P T T T T T T T
T T T P P P P P P T T T T T T T
T T T P P P P P P P T T T T T T
T T P P P P P P P P P P P P P P
T T P P P T T T P P P P P P P P
T T T T T T T T P P P P P P P P
T T T T T T T P P P T P P P P P
T T T T T T P P P P T P P P P P
T T T T T T P P P P T P P P P P
T T T T P P P P P P R R R R R P
T T T P P P P P P P R A A A R P
T T T T P P P P P P R A A A R P
T T T T P T T P P P R R R R R P
T T T T P T T P P P P P T T P P
T T T T P P P P P P P P T T T P
```

El cargador de Assimp separa la malla en un *mesh* por material, así que cada zona se dibuja con su propia textura sin cambios adicionales en el código.

7. Actividad 6: acomodo de los árboles

El árbol tiene la base de su tronco en `y = 0` (0.28 × 0.28 unidades con escala 2). Las cinco instancias originales estaban todas en `y = 0`, sin importar el relieve. Se calculó la altura del terreno bajo cada tronco (intersección con los triángulos de `Floor.obj`) y se usó la **altura mínima** bajo el tronco, para que ninguna parte quede flotando. También se verificó que la copa no choque con el terreno. Las instancias se guardan en el arreglo `posArboles[9]`.

Árbol	Posición (x, y, z)	Estado original
1 (original)	(0, 0.347, 8)	Hundido 0.35
2 (original)	(8, 0, 0)	Correcto
3 (original)	(−8, 0.169, 0)	Hundido 0.17
4 (original)	(0, 0, −10)	Correcto
5 (original)	(3, 0, −15)	Correcto
6 (nuevo)	(−6, 0, −8)	—
7 (nuevo)	(10, 0, −9)	—
8 (nuevo)	(12, 0, 6)	—
9 (nuevo)	(−10, 0.167, −14), sobre una loma	—

8. Mejoras al cargador de modelos

Para integrar modelos descargados de distintas fuentes sin errores, se hicieron estos cambios en `Model.h` y `Mesh.h`:

- **Normales:** se agregó `aiProcess_GenSmoothNormals`, y además se verifica `mNormals`. Antes, un modelo sin normales cerraba el programa.
- **Color de material:** si un modelo no tiene textura, como muchos modelos low-poly, el shader usa su color difuso (uniformes `usarTextura` y `colorDifuso`) en vez de dibujarse negro.
- **Texturas incrustadas:** las rutas `*N` de archivos GLB y FBX (Mixamo) se cargan desde memoria con `SOIL_load_image_from_memory`.
- **Rutas absolutas:** si la textura no está en la ruta indicada por el modelo, se busca solo por su nombre de archivo en la carpeta del modelo.
- **Modelos estáticos:** la opción `preTransform` aplica las transformaciones de los nodos (`aiProcess_PreTransformVertices`), y se calcula la caja envolvente del modelo.
- **Colocación automática:** `ColocarModelo()` escala cada modelo a una altura en unidades de la escena, lo centra y apoya su base sobre el terreno. La altura del piso se obtiene con `AlturaTerreno(x, z)` por coordenadas baricéntricas.

9. Controles

Tecla	Acción
W / A / S / D o flechas	Mover la cámara (adelante, izquierda, atrás, derecha)
Mouse	Girar la cámara
L	Cambiar la luz direccional entre blanco y naranja (intensidad 0.5)
Z	Zorro: iniciar o detener la caminata
X	Zorro: saltar
C	Pausar o reanudar la animación en bucle de la campana
R	Encender o apagar la spotlight roja
V	Encender o apagar la spotlight verde
B	Encender o apagar la spotlight azul
Esc	Salir del programa

Las demás animaciones (tren, cañón, bandera, jefe revolucionario y Adelita) se reproducen automáticamente en bucle.

> El PDF del examen asigna la tecla **V** tanto a "reanudar animación" (C/V) como a "luz verde" (R/V/B). Para que ambas funciones operen sin conflicto, **C alterna pausar/reanudar** la animación en bucle y **V** controla la luz verde. Además, la animación de Steve (solo referencia del proyecto base) se movió de B a N, porque B es la luz azul.

10. Compilación y ejecutable

- Configuraciones **Debug | x86** y **Release | x86**. Las librerías incluidas son de 32 bits.
- La librería `soil2-debug.lib` enlaza el runtime de depuración (`MSVCRTD`), que solo existe en equipos con Visual Studio. Por eso, en Release se compilan directamente los fuentes de SOIL2 (`SOIL2/*.c`, ya incluidos en el proyecto) y no se enlaza esa librería. Se verificó que el ejecutable Release no depende de ninguna DLL de depuración.
- El paquete incluye `ExtraOrdinario.exe`, `assimp-vc140-mt.dll`, `glew32.dll`, `vcruntime140.dll` y `msvcp140.dll` (redistribuibles de Visual C++), más las carpetas `Shader`, `SkyBox` y `ModelsExtra`.
- El instalador `Instalador_ExtraordinarioCG.exe` se generó con IExpress, la herramienta incluida en Windows. Copia el programa a `%LOCALAPPDATA%\ExtraordinarioCG` y crea un acceso directo en el Escritorio. También se entrega la versión portable `ExtraordinarioCG.zip`.

11. Capturas de la escena

Vista general del pueblo: catedral, campanario, casas, caballo, barriles, lago y vía del tren.



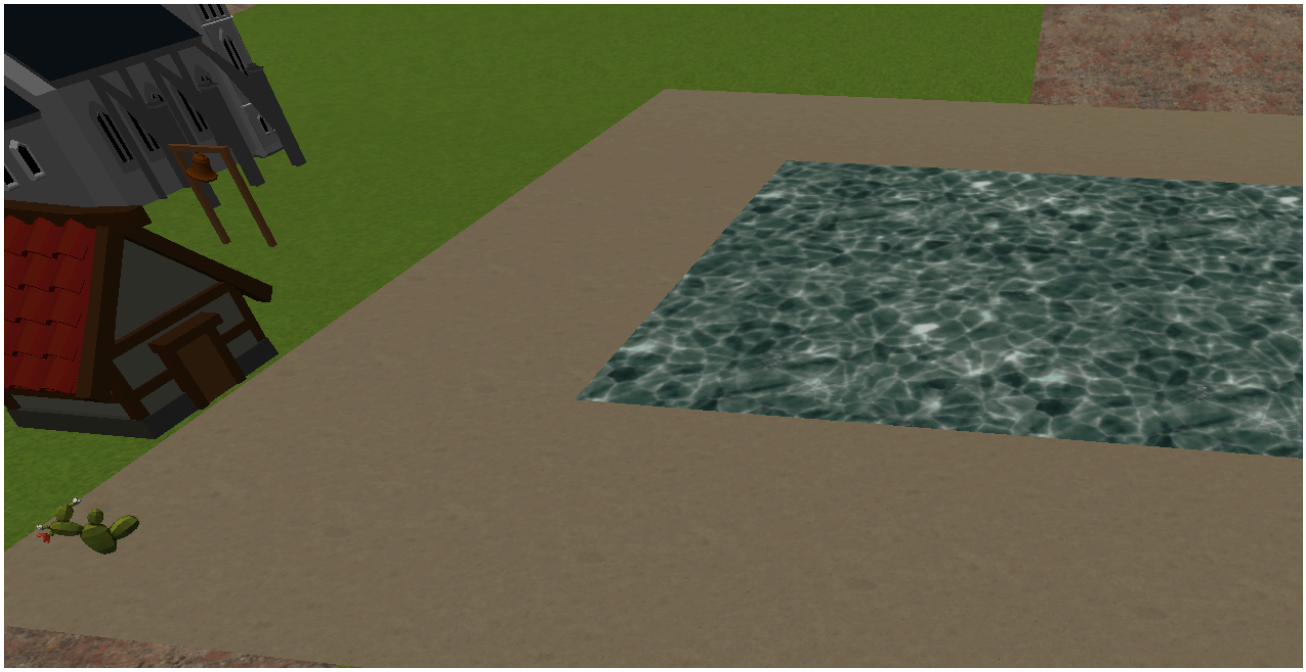
Spotlights RGB sobre el jefe revolucionario (cubos rojo, verde y azul) en la plaza.



Tren sobre la vía (bucle 1), con la bandera, las casas y la catedral al fondo.



Piso con 4 texturas: agua, arena alrededor, pasto y tierra en el camino.



Zorro, nopal, cañón y Adelita; tierra en los cerros.



12. Referencias

Modelos 3D

Adobe. (s.f.). *Peasant Girl* [Personaje 3D] y *Excited* [Animación]. Mixamo. Licencia de uso libre de Mixamo. https://www.mixamo.com

Adobe. (s.f.). *The Boss* [Personaje 3D] y *Reloading* [Animación]. Mixamo. Licencia de uso libre de Mixamo. https://www.mixamo.com

CreativeTrio. (s.f.). *Rifle* [Modelo 3D, licencia CC0]. Poly Pizza. https://poly.pizza/m/zBATGsIN2h

Oliveira, B. (s.f.). *Cathedral* [Modelo 3D, licencia CC BY 3.0]. Poly Pizza. https://poly.pizza/m/fEJKTKNRAsN

Poly by Google. (s.f.). *Curved road* [Modelo 3D, licencia CC BY 3.0]. Poly Pizza. https://poly.pizza/m/9r2Owu4oVHw

Poly by Google. (s.f.). *Prickly pear cactus* [Modelo 3D, licencia CC BY 3.0]. Poly Pizza. https://poly.pizza/m/47Af27jQemx

Poly by Google. (s.f.). *Sombrero* [Modelo 3D, licencia CC BY 3.0]. Poly Pizza. https://poly.pizza/m/7fGDqHvHap1

Quaternius. (s.f.). *Barrel* [Modelo 3D, licencia CC0]. Poly Pizza. https://poly.pizza/m/zjCQP1TAci

Quaternius. (s.f.). *Bell* [Modelo 3D, licencia CC0]. Poly Pizza. https://poly.pizza/m/cILuaO115Y

Quaternius. (s.f.). *Cannon* [Modelo 3D, licencia CC0]. Poly Pizza. https://poly.pizza/m/J15vlPVvKK

Quaternius. (s.f.). *Goal Flag* [Modelo 3D, licencia CC0]. Poly Pizza. https://poly.pizza/m/onAi7dx56P

Quaternius. (s.f.). *Horse* [Modelo 3D, licencia CC0]. Poly Pizza. https://poly.pizza/m/qvTrSG9pZF

Quaternius. (s.f.). *House* [Modelo 3D, licencia CC0]. Poly Pizza. https://poly.pizza/m/oJJIRwv6Bo

Quaternius. (s.f.). *Locomotive Front* [Modelo 3D, licencia CC0]. Poly Pizza. https://poly.pizza/m/WY84FHug9s

Quaternius. (s.f.). *Locomotive Passenger Carriage* [Modelo 3D, licencia CC0]. Poly Pizza. https://poly.pizza/m/woQPFlzPHM

Texturas

ambientCG. (s.f.). *Grass005* [Textura, licencia CC0]. https://ambientcg.com/view?id=Grass005

ambientCG. (s.f.). *Ground054* [Textura, licencia CC0]. https://ambientcg.com/view?id=Ground054

ambientCG. (s.f.). *Ground111* [Textura, licencia CC0]. https://ambientcg.com/view?id=Ground111

ambientCG. (s.f.). *Ice002* [Textura, licencia CC0]. https://ambientcg.com/view?id=Ice002

Software, tutoriales y repositorios

Assimp Team. (2017). *Open Asset Import Library* (Versión 4.1.0) [Software]. https://github.com/assimp/assimp

de Vries, J. (2020). *Learn OpenGL: Learn modern OpenGL graphics programming in a structured manner*. https://learnopengl.com

GLFW Development Team. (s.f.). *GLFW* [Software]. https://www.glfw.org

joserapath23. (2026). *Extraordinario2027-1* [Repositorio de código fuente]. GitHub. https://github.com/joserapath23/Extraordinario2027-1

Khronos Group. (2017). *glTF 2.0 Specification*. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html